

ML-Ops Experimental Learning Assignment

End-to-End ML Model Development, CI/CD, and Production Deployment Experimental Learning

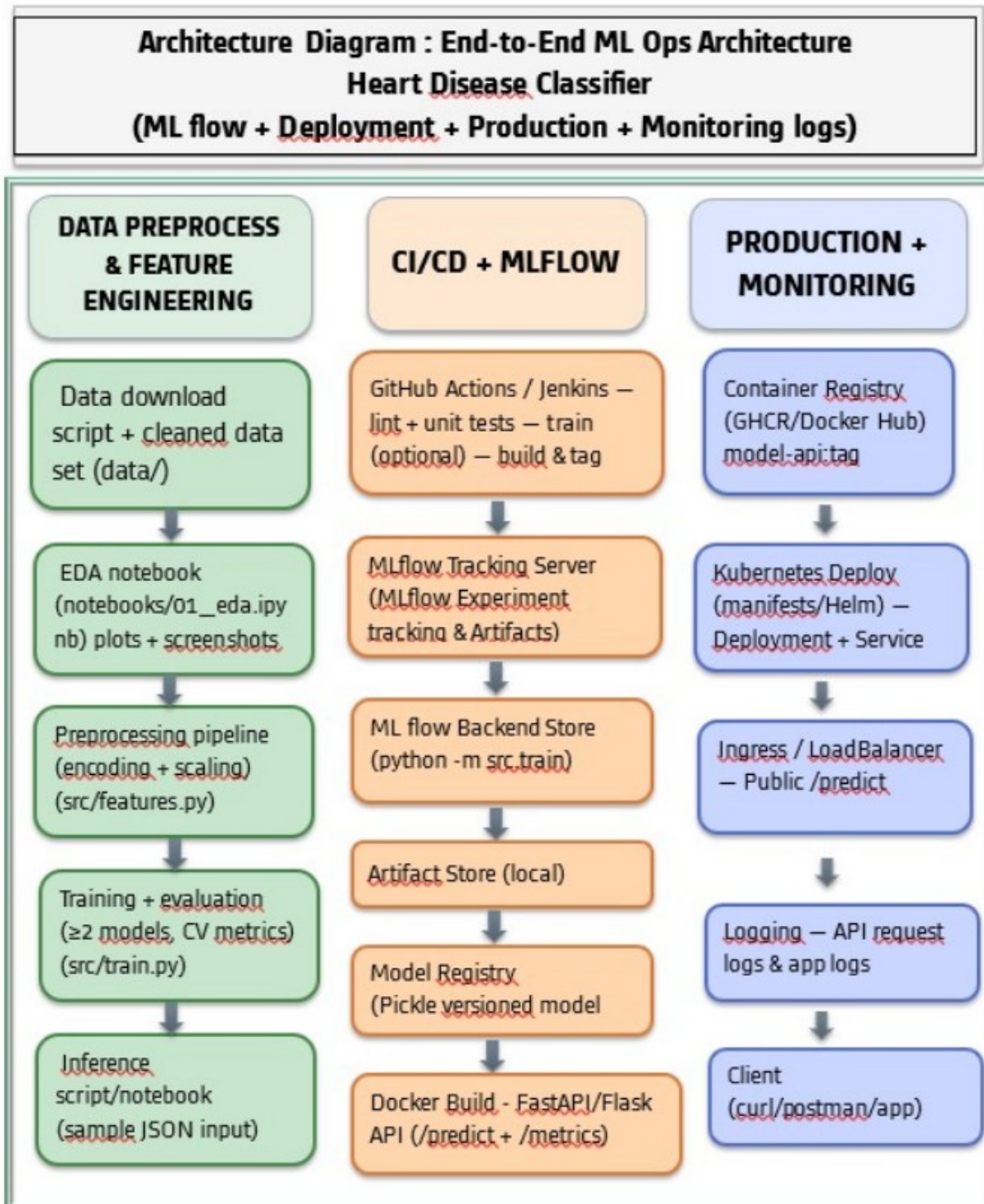
Objective:

Design, develop, and deploy a scalable and reproducible machine learning solution utilising modern MLOps best practices. The assignment emphasises practical automation, experiment tracking, CI/CD pipelines, containerization, cloud deployment, and monitoring—mirroring real-world production scenarios.

Team & Group Details: Group No - 3

S. No	Name	BITS ID	Contribution
1	Karan Sharma	2024AB05145	100%
2	Selva Pandian	2023AC05005	100%
3	Neerumalla Kavitha	2024AA05879	100%
4	Suraj Anand	2024AA05731	100%
5	Shikhar Nigam	2024AA05691	100%

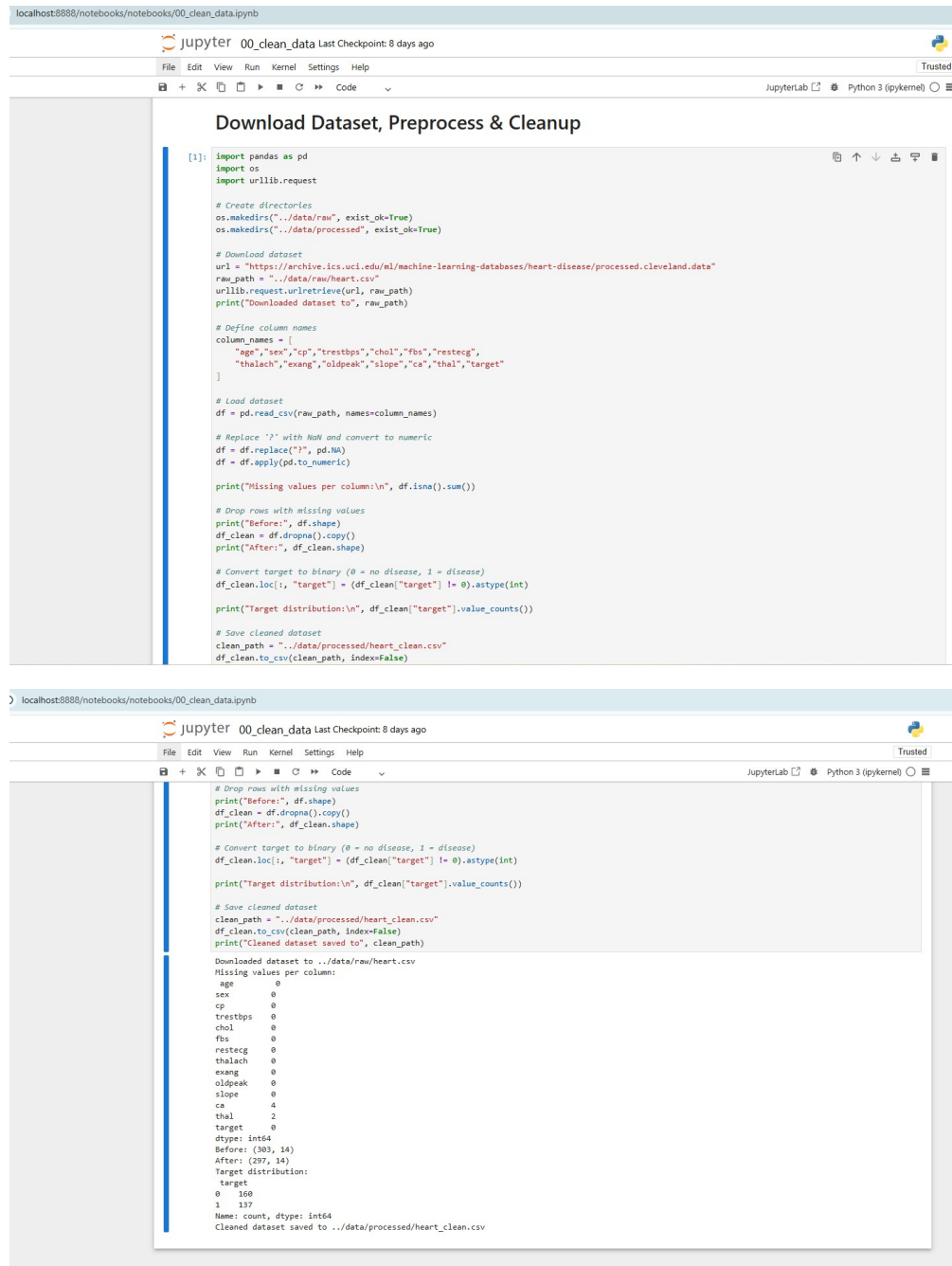
Architecture Diagram:



Assignment Tasks

1. Data Acquisition & Exploratory Data Analysis (EDA) [5 marks]

Obtain the dataset (provide download script or instructions). Clean and preprocess the data (handle missing values, encode features). Perform EDA with professional visualizations (histograms, correlation heatmaps, class balance).



The image displays two screenshots of a Jupyter Notebook interface, showing the process of downloading, cleaning, and preprocessing a dataset.

Top Screenshot: Download Dataset, Preprocess & Cleanup

```
[1]: import pandas as pd
import os
import urllib.request

# Create directories
os.makedirs("../data/raw", exist_ok=True)
os.makedirs("../data/processed", exist_ok=True)

# Download dataset
url = "https://archive.ics.uci.edu/ml/machine-learning-databases/heart-disease/processed.cleveland.data"
raw_path = "../data/raw/heart.csv"
urllib.request.urlretrieve(url, raw_path)
print("Downloaded dataset to", raw_path)

# Define column names
column_names = [
    "age", "sex", "cp", "trestbps", "chol", "fbs", "restecg",
    "thalach", "exang", "oldpeak", "slope", "ca", "thal", "target"
]

# Load dataset
df = pd.read_csv(raw_path, names=column_names)

# Replace '?' with NaN and convert to numeric
df = df.replace("?", pd.NA)
df = df.apply(pd.to_numeric)

print("Missing values per column:\n", df.isna().sum())

# Drop rows with missing values
print("Before:", df.shape)
df_clean = df.dropna().copy()
print("After:", df_clean.shape)

# Convert target to binary (0 = no disease, 1 = disease)
df_clean.loc[:, "target"] = (df_clean["target"] != 0).astype(int)

print("Target distribution:\n", df_clean["target"].value_counts())

# Save cleaned dataset
clean_path = "../data/processed/heart_clean.csv"
df_clean.to_csv(clean_path, index=False)
```

Bottom Screenshot: Output of the cleaning process

```
# Drop rows with missing values
print("Before:", df.shape)
df_clean = df.dropna().copy()
print("After:", df_clean.shape)

# Convert target to binary (0 = no disease, 1 = disease)
df_clean.loc[:, "target"] = (df_clean["target"] != 0).astype(int)

print("Target distribution:\n", df_clean["target"].value_counts())

# Save cleaned dataset
clean_path = "../data/processed/heart_clean.csv"
df_clean.to_csv(clean_path, index=False)
print("Cleaned dataset saved to", clean_path)

Downloaded dataset to ../data/raw/heart.csv
Missing values per column:
age      0
sex      0
cp       0
trestbps 0
chol     0
fbs      0
restecg  0
thalach  0
exang    0
oldpeak  0
slope    0
ca       4
thal     2
target   0
dtype: int64
Before: (303, 14)
After: (297, 14)
Target distribution:
target
0    160
1    137
Name: count, dtype: int64
Cleaned dataset saved to ../data/processed/heart_clean.csv
```

Exploratory Data Analysis (EDA)

Objective: Understand data distribution, feature relationships, and target balance before model development.

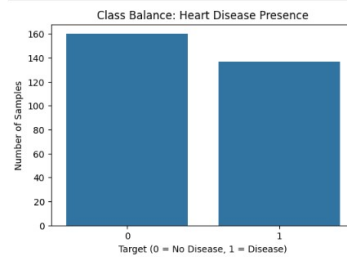
1. Class Balance Visualization :

The dataset shows a relatively balanced distribution between patients with and without heart disease. Therefore, no additional class imbalance handling techniques (such as SMOTE) are required at this stage.

```
[2]: import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

df = pd.read_csv("../data/processed/heart_clean.csv")
df.head()

plt.figure(figsize=(6,4))
sns.countplot(x="target", data=df)
plt.title("Class Balance: Heart Disease Presence")
plt.xlabel("Target (0 = No Disease, 1 = Disease)")
plt.ylabel("Number of Samples")
plt.show()
```

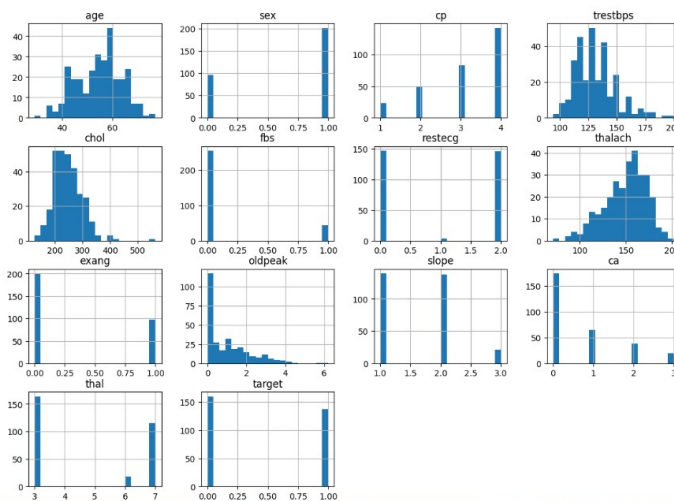


2. Feature Distribution – Histograms :

Most numerical features such as age, resting blood pressure, and cholesterol follow approximately normal distributions, while features like fasting blood sugar and exercise-induced angina are skewed. This motivates feature scaling during model training.

```
[3]: df.hist(figsize=(14,10), bins=20)
plt.suptitle("Feature Distributions (Histograms)", fontsize=16)
plt.show()
```

Feature Distributions (Histograms)



localhost8888/notebooks/notebooks/01_eda.ipynb

jupyter01_edaLast Checkpoint: 11 hours ago

FileEditViewRunKernelSettingsHelp

Python 3 (ipykernel)

Trusted

3. Correlation Heatmap

Chest pain type, maximum heart rate achieved, and exercise-induced angina show strong correlation with the target variable. No extreme multicollinearity is observed, making the dataset suitable for classical machine learning models.

```
[4]: plt.figure(figsize=(12,8))
sns.heatmap(
    df.corr(),
    cmap="coolwarm",
    linewidths=0.5
)
plt.title("Correlation Heatmap of Features")
plt.show()
```

Correlation Heatmap of Features

age sex cp trestbps chol fbs restecg thalach exang oldpeak slope ca thal target

EDA Summary

- The dataset is clean with minimal missing values
- Target classes are reasonably balanced
- Feature distributions suggest the need for scaling
- Several clinical features show meaningful correlation with heart disease presence
- These insights guide preprocessing and model selection in subsequent stages

[]:

2. Feature Engineering & Model Development [8 marks]

Prepare the final ML features (scaling and encoding). Build and train at least two classification models (e.g., Logistic Regression and Random Forest). Document model selection and tuning process. Evaluate using cross-validation and relevant metrics (accuracy, precision, recall, ROC-AUC).

```
train.py × preprocess.py × 00_clean_data.py application.yml configmap.yml deployment.yml prod.env

25 def find_missing_values(df):  # KSharma-SourceCode
26     return (df == "?").sum()
27
28 def fix_target(df):  # KSharma-SourceCode
29     df = df.copy()
30     df["target"] = df["target"].apply(lambda x: 1 if x > 0 else 0)
31     return df
32
33 def build_preprocessor():  # 1 usage (1 dynamic) # KSharma-SourceCode
34     numeric_features = [
35         "age", "trestbps", "chol", "thalach", "oldpeak"
36     ]
37
38     categorical_features = [
39         "sex", "cp", "fbs", "restecg",
40         "exang", "slope", "ca", "thal"
41     ]
42
43     numeric_transformer = Pipeline(steps=[
44         ("scaler", StandardScaler())
45     ])
46
47     categorical_transformer = Pipeline(steps=[
48         ("encoder", OneHotEncoder(handle_unknown="ignore"))
49     ])
50
51     preprocessor = ColumnTransformer(
52         transformers=[
53             ("num", numeric_transformer, numeric_features),
54             ("cat", "passthrough", categorical_features)
55         ]
56     )
57
58     return preprocessor
59
60 def split_features_target(df):  # KSharma-SourceCode
61     X = df.drop(columns=["target"])
62     y = df["target"]
63     return X, y
```

```
Terminal Local x + v
(.venv) PS G:\Karan\GitHub\AI-ML-Assignments\M-Tech. Semester 3\MLOPS\Assignment-1\heart-disease-mlops>
(.venv) PS G:\Karan\GitHub\AI-ML-Assignments\M-Tech. Semester 3\MLOPS\Assignment-1\heart-disease-mlops> python -m src.train
2026/01/05 21:45:55 INFO mlflow.store.db.utils: Creating initial MLflow database tables...
2026/01/05 21:45:55 INFO mlflow.store.db.utils: Updating database tables
2026/01/05 21:45:55 INFO alembic.runtime.migration: Context impl SQLiteImpl.
2026/01/05 21:45:55 INFO alembic.runtime.migration: Will assume non-transactional DDL.
2026/01/05 21:45:55 INFO alembic.runtime.migration: Context impl SQLiteImpl.
2026/01/05 21:45:55 INFO alembic.runtime.migration: Will assume non-transactional DDL.
CV LogisticRegression accuracy: 0.8227
CV LogisticRegression precision: 0.8203
CV LogisticRegression recall: 0.8043
CV LogisticRegression roc_auc: 0.8810
CV LogisticRegression f1: 0.8112
LogisticRegression Metrics:
  Accuracy : 0.9000
  Precision : 0.8750
  Recall : 0.8750
  F1-Score : 0.8750
  ROC-AUC : 0.9468
CV RandomForest accuracy: 0.7972
CV RandomForest precision: 0.8133
CV RandomForest recall: 0.7423
CV RandomForest roc_auc: 0.8779
CV RandomForest f1: 0.7753
RandomForest Metrics:
  Accuracy : 0.8667
  Precision : 0.8636
  Recall : 0.7917
  F1-Score : 0.8261
  ROC-AUC : 0.9306

Best Model: LogisticRegression with Accuracy: 0.9000
Model saved at: model/development/v2/heart_model.pkl
✔ Model v2 promoted to production
(.venv) PS G:\Karan\GitHub\AI-ML-Assignments\M-Tech. Semester 3\MLOPS\Assignment-1\heart-disease-mlops>
```

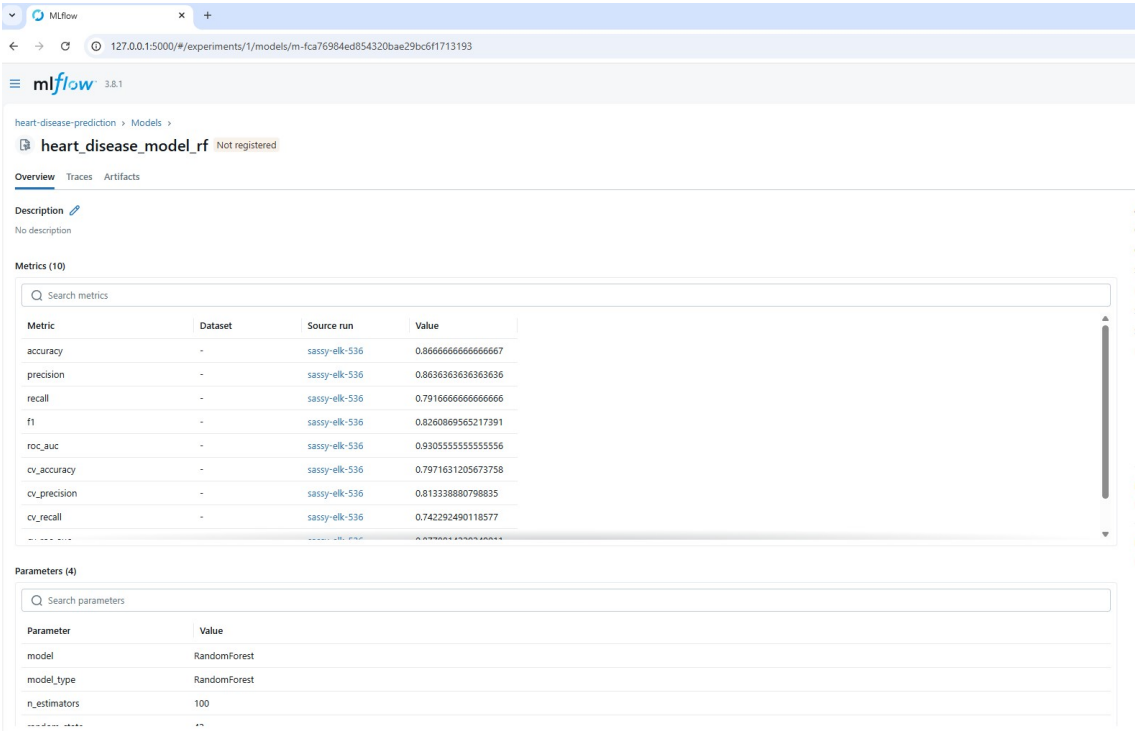
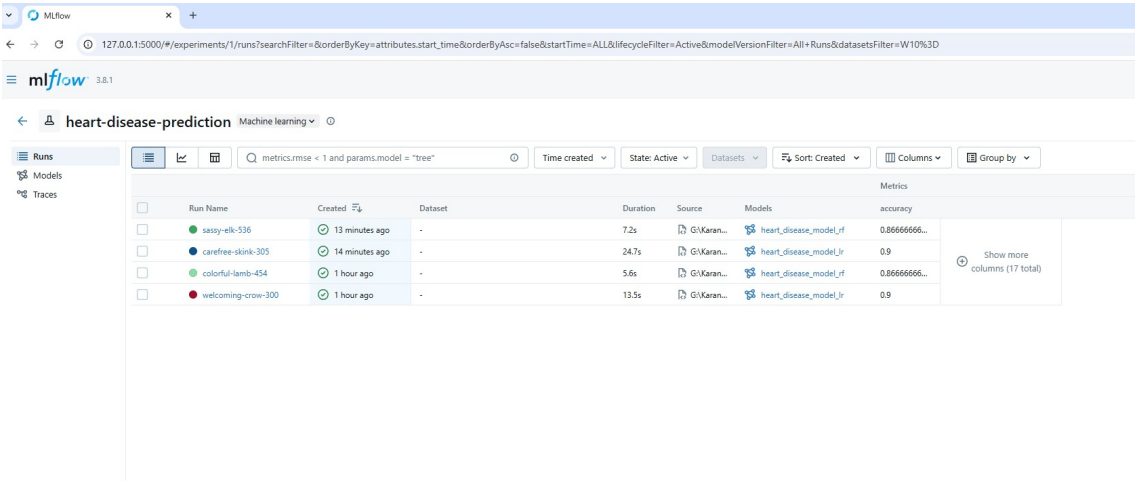
```
train.py x preprocess.py 00_clean_data.py application.yml configmap.yml deployment.yml prod.env dev.env
> Q: evaluate_model 1/3
137
138 def train_logistic_regression(X_train, y_train, X_test, y_test, version, env): 1 usage KSharma-SourceCode
139     with mlflow.start_run():
140         mlflow.log_params({
141             "model": "LogisticRegression",
142             "max_iter": 1000,
143             "random_state": 42
144         })
145
146         pipeline = Pipeline(steps=[
147             ("preprocessor", build_preprocessor()),
148             ("model", LogisticRegression(max_iter=1000, random_state=42))
149         ])
150
151         pipeline.fit(X_train, y_train)
152         acc = evaluate_model(pipeline, X_train, y_train, X_test, y_test, model_name="LogisticRegression", version, env)
153
154         mlflow.sklearn.log_model(sk_model=pipeline, name="heart_disease_model_lr")
155         return pipeline, acc
156
157 def train_random_forest(X_train, y_train, X_test, y_test, version, env): 1 usage KSharma-SourceCode
158     with mlflow.start_run():
159         mlflow.log_params({
160             "model": "RandomForest",
161             "n_estimators": 100,
162             "random_state": 42
163         })
164
165         pipeline = Pipeline(steps=[
166             ("preprocessor", build_preprocessor()),
167             ("model", RandomForestClassifier(n_estimators=100, random_state=42))
168         ])
169
170         pipeline.fit(X_train, y_train)
171         acc = evaluate_model(pipeline, X_train, y_train, X_test, y_test, model_name="RandomForest", version, env)
172
173         mlflow.sklearn.log_model(sk_model=pipeline, name="heart_disease_model_rf")
174         return pipeline, acc
175
176 def promote_model_to_production(version): 1 usage KSharma-SourceCode
```



```
train.py × preprocess.py 00_clean_data.py application.yml configmap.yml deployment.yml prod.env dev.env
> Q- evaluate_model × ↻ Cc W .* 1/3 ↑ ↓ 🔍 :
180
181 def evaluate_model(pipeline, X_train, y_train, X_test, y_test, model_name, version, env): 2 usages 🔍 KSharma-SourceCode
182     # Predictions
183     y_pred = pipeline.predict(X_test)
184     y_proba = pipeline.predict_proba(X_test)[:, 1] if hasattr(pipeline, "predict_proba") else None
185
186     # Metrics
187     acc = accuracy_score(y_test, y_pred)
188     precision = precision_score(y_test, y_pred)
189     recall = recall_score(y_test, y_pred)
190     roc_auc = roc_auc_score(y_test, y_proba) if y_proba is not None else None
191     f1 = f1_score(y_test, y_pred)
192
193     # Log to MLflow
194     mlflow.log_param(key="model_type", value=model_name)
195     mlflow.log_metric(key="accuracy", value=acc)
196     mlflow.log_metric(key="precision", value=precision)
197     mlflow.log_metric(key="recall", value=recall)
198     mlflow.log_metric(key="f1", value=f1)
199     if roc_auc is not None:
200         mlflow.log_metric(key="roc_auc", value=roc_auc)
201
202     # Log metrics and save artifacts
203     log_confusion_matrix(y_test, y_pred, model_name, version, env)
204     log_classification_report(y_test, y_pred, model_name, version, env)
205     log_cross_validation_metrics(pipeline, X_train, y_train, model_name, version, env)
206
207     # Console output
208     print(f"{model_name} Metrics:")
209     print(f" Accuracy : {acc:.4f}")
210     print(f" Precision : {precision:.4f}")
211     print(f" Recall : {recall:.4f}")
212     print(f" F1-Score : {f1:.4f}")
213     if roc_auc is not None:
214         print(f" ROC-AUC : {roc_auc:.4f}")
215
216     return acc
217
218 def train_logistic_regression(X_train, y_train, X_test, y_test, version, env): 1 usage 🔍 KSharma-SourceCode
219     with mlflow.start_run():
```

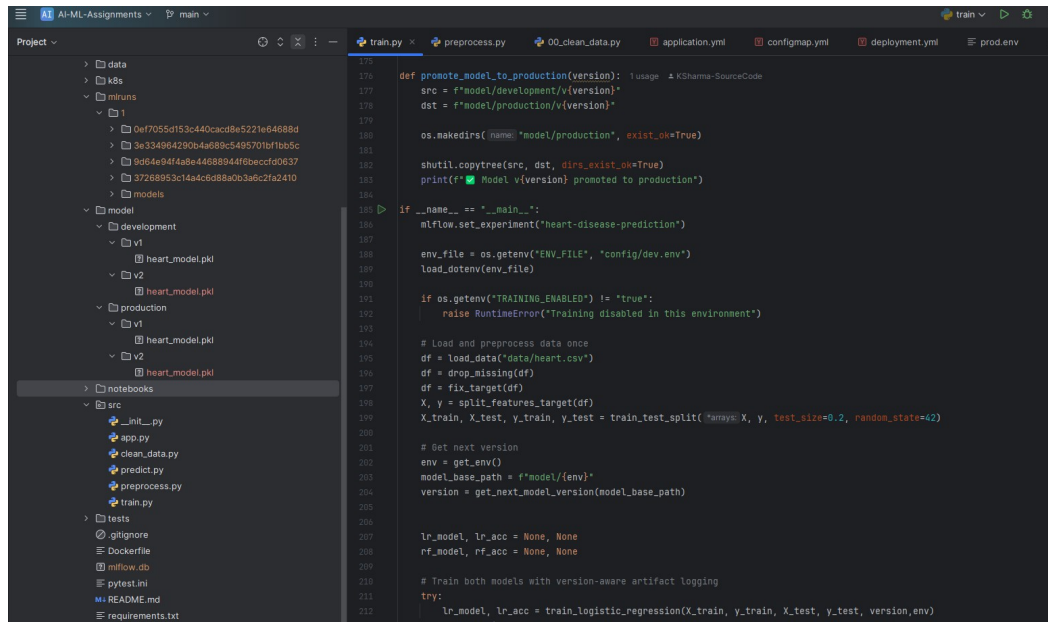

3. Experiment Tracking [5 marks]

Integrate MLflow (or a similar tool) for experiment tracking. Log parameters, metrics, artifacts, and plots for all runs. MLOps (S1-25_AIMLCZG523)



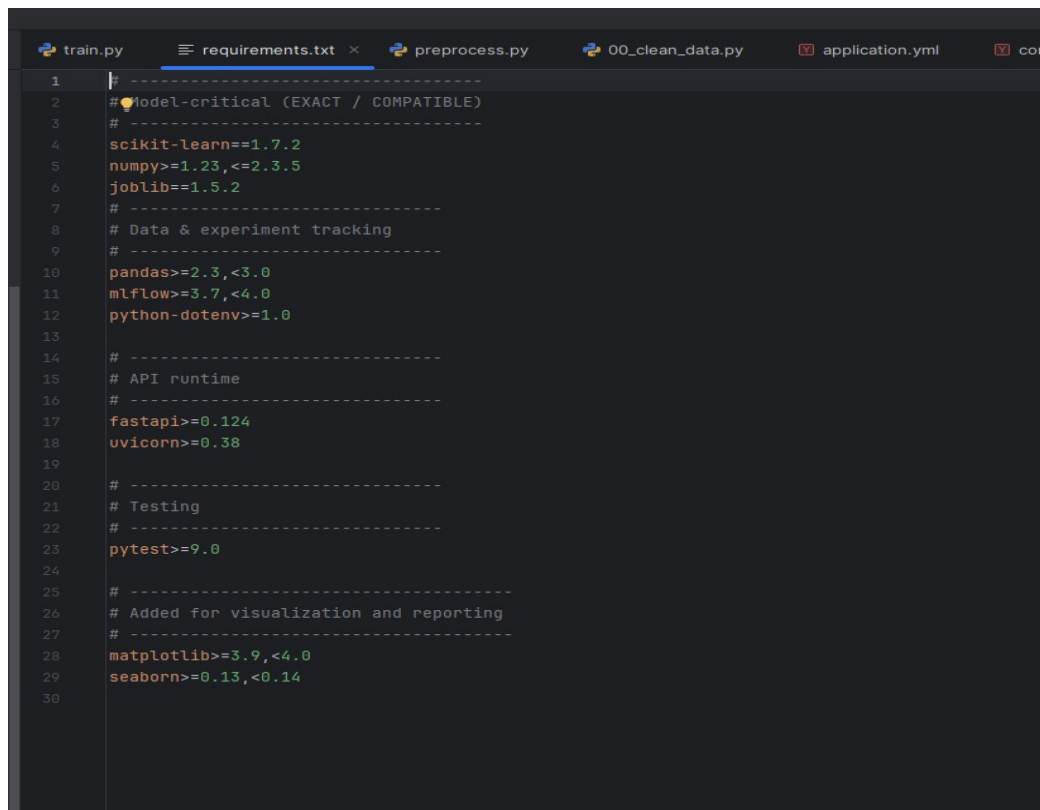
4. Model Packaging & Reproducibility [7 marks]

Save the final model in a reusable format (e.g., MLflow, pickle, ONNX). Write a clean requirements.txt (or Conda env file). Provide a preprocessing pipeline/transformers to ensure full reproducibility.



The screenshot shows a code editor with a project structure on the left and a Python script on the right. The project structure includes folders for data, notebooks, src, and tests, and files for requirements.txt, Dockerfile, mlflow.db, pytest.ini, README.md, and requirements.txt. The Python script, named train.py, defines a function promote_model_to_production(version) that promotes a model from development to production. It uses os.makedirs to create the production directory, shutil.copytree to copy the model files, and print to log the promotion. The script also includes a main function that sets up the MLflow experiment, loads the environment file, and trains the model. The script is written in Python 3.7 and uses the following dependencies: pandas, numpy, joblib, mlflow, and python-dotenv.

```
175
176 def promote_model_to_production(version):
177     src = f"model/development/v{version}"
178     dst = f"model/production/v{version}"
179
180     os.makedirs(dst, exist_ok=True)
181
182     shutil.copytree(src, dst, dirs_exist_ok=True)
183     print(f"Model v{version} promoted to production")
184
185 if __name__ == "__main__":
186     mlflow.set_experiment("heart-disease-prediction")
187
188     env_file = os.getenv("ENV_FILE", "config/dev.env")
189     load_dotenv(env_file)
190
191     if os.getenv("TRAINING_ENABLED") != "true":
192         raise RuntimeError("Training disabled in this environment")
193
194     # Load and preprocess data once
195     df = load_data("data/heart.csv")
196     df = drop_missing(df)
197     df = fix_target(df)
198     X, y = split_features_target(df)
199     X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
200
201     # Get next version
202     env = get_env()
203     model_base_path = f"model/{env}"
204     version = get_next_model_version(model_base_path)
205
206     lr_model, lr_acc = None, None
207     rf_model, rf_acc = None, None
208
209     # Train both models with version-aware artifact logging
210     try:
211         lr_model, lr_acc = train_logistic_regression(X_train, y_train, X_test, y_test, version, env)
212         rf_model, rf_acc = train_random_forest(X_train, y_train, X_test, y_test, version, env)
```

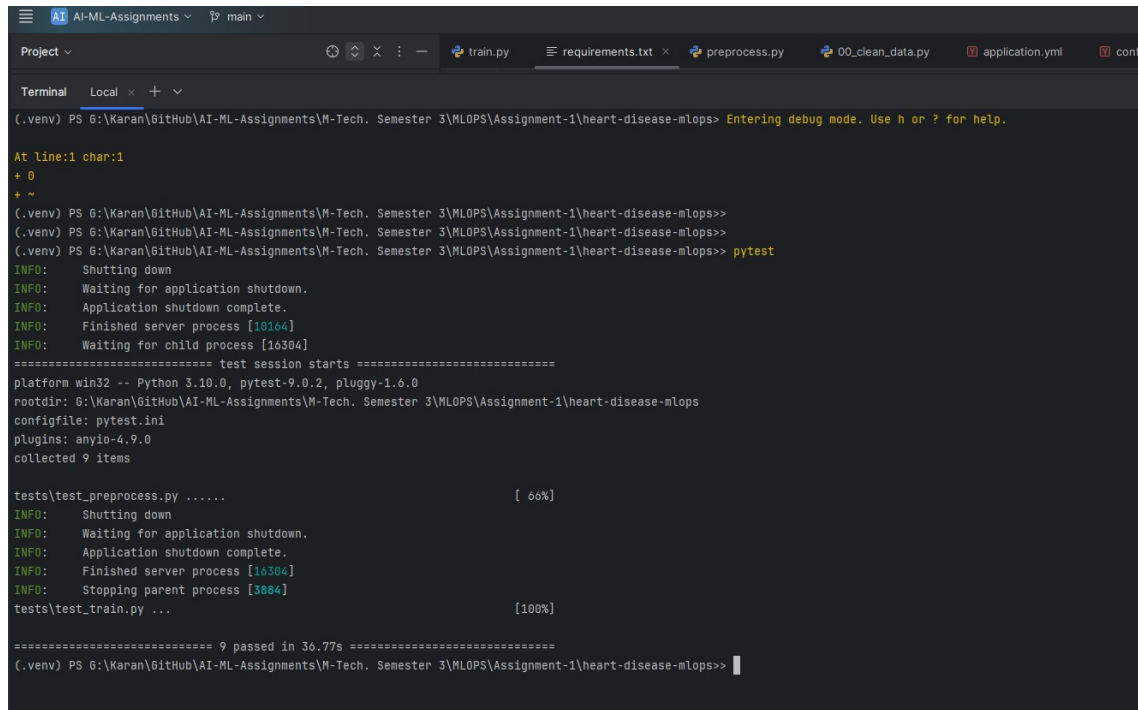


The screenshot shows a code editor with a requirements.txt file. The file contains the following dependencies and versions:

```
1 # -----
2 # Model-critical (EXACT / COMPATIBLE)
3 # -----
4 scikit-learn==1.7.2
5 numpy>=1.23,<=2.3.5
6 joblib==1.5.2
7 # -----
8 # Data & experiment tracking
9 # -----
10 pandas>=2.3,<3.0
11 mlflow>=3.7,<4.0
12 python-dotenv>=1.0
13 # -----
14 # API runtime
15 # -----
16 fastapi>=0.124
17 uvicorn>=0.38
18 # -----
19 # Testing
20 # -----
21 pytest>=9.0
22 # -----
23 # Added for visualization and reporting
24 # -----
25 matplotlib>=3.9,<4.0
26 seaborn>=0.13,<0.14
27 # -----
28 # -----
29 # -----
30 # -----
```

5. CI/CD Pipeline & Automated Testing [8 marks]

Write unit tests for data processing and model code (Pytest or unit test). Create a GitHub Actions (or Jenkins) pipeline That Includes Linting, unit testing, and model training steps. Artifacts/logging for each workflow run.



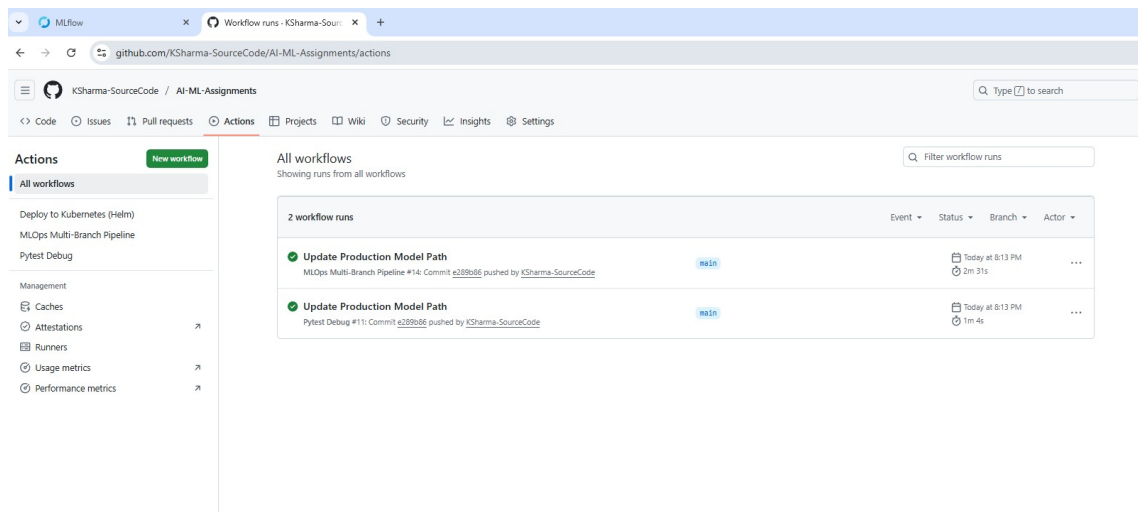
```
(.venv) PS G:\Karan\GitHub\AI-ML-Assignments\M-Tech. Semester 3\MLOPS\Assignment-1\heart-disease-mlops> Enter debug mode. Use h or ? for help.

At line:1 char:1
+ 0
+ ~

(.venv) PS G:\Karan\GitHub\AI-ML-Assignments\M-Tech. Semester 3\MLOPS\Assignment-1\heart-disease-mlops>>
(.venv) PS G:\Karan\GitHub\AI-ML-Assignments\M-Tech. Semester 3\MLOPS\Assignment-1\heart-disease-mlops>>
(.venv) PS G:\Karan\GitHub\AI-ML-Assignments\M-Tech. Semester 3\MLOPS\Assignment-1\heart-disease-mlops>> pytest
INFO: Shutting down
INFO: Waiting for application shutdown.
INFO: Application shutdown complete.
INFO: Finished server process [10164]
INFO: Waiting for child process [16304]
===== test session starts =====
platform win32 -- Python 3.10.0, pytest-9.0.2, pluggy-1.6.0
rootdir: G:\Karan\GitHub\AI-ML-Assignments\M-Tech. Semester 3\MLOPS\Assignment-1\heart-disease-mlops
configfile: pytest.ini
plugins: anyio-4.9.0
collected 9 items

tests\test_preprocess.py ..... [ 66%]
INFO: Shutting down
INFO: Waiting for application shutdown.
INFO: Application shutdown complete.
INFO: Finished server process [16304]
INFO: Stopping parent process [5884]
tests\test_train.py ... [100%]

===== 9 passed in 36.77s =====
(.venv) PS G:\Karan\GitHub\AI-ML-Assignments\M-Tech. Semester 3\MLOPS\Assignment-1\heart-disease-mlops>> |
```



MLflow

Update Production Model Path

github.com/KSharma-SourceCode/AI-ML-Assignments/actions/runs/20718916807/job/59477071466

KSharma-SourceCode / AI-ML-Assignments

<> Code

Issues

Pull requests

Actions

Projects

Wiki

Security

Insights

Settings

← MLOps Multi-Branch Pipeline

Update Production Model Path #14

Summary

All jobs

mlops

Run details

Usage

Workflow file

mlops

succeeded 1 hour ago in 2m 28s

> Set up job

> Checkout code

> Set up Python

> Install dependencies

> Run Unit Tests

> Detect ML Code Changes

Train model (development only if ML code changed)

Train model (staging)

> Train model (production)

> Build Docker image

> Docker login

> Build Docker image

> Push image to Docker Hub

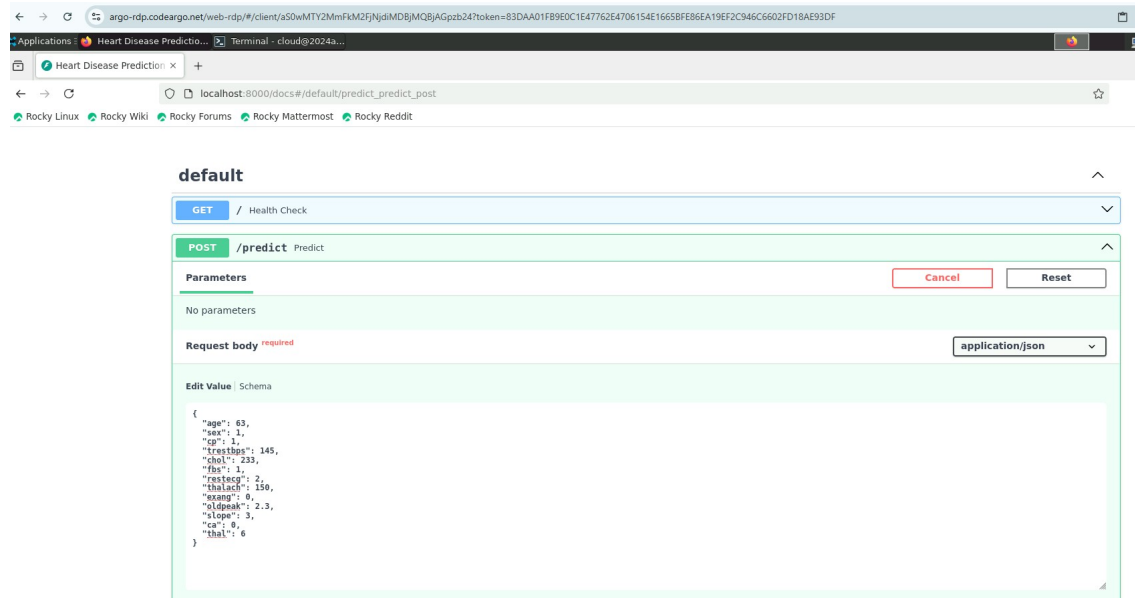
> Post Set up Python

> Post Checkout code

> Complete job

6. Model Containerization [5 marks]

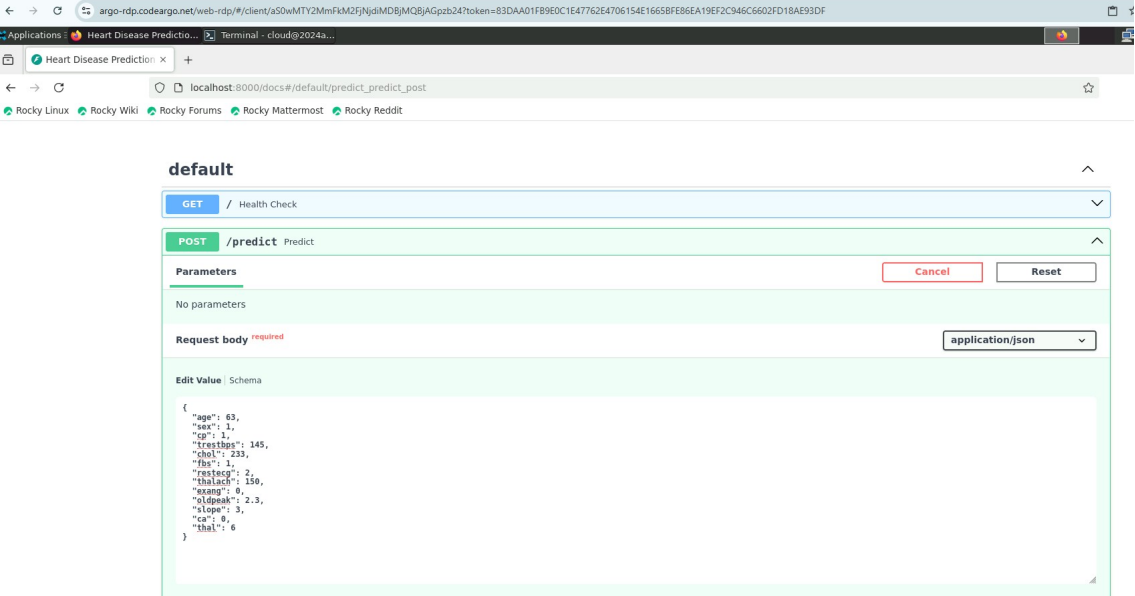
Build a Docker container for the model-serving API (Flask or FastAPI is recommended).
Expose /predict endpoint, accept JSON input, return prediction and confidence. The container must be built and run locally with the sample input.



7. Production Deployment [7 marks]

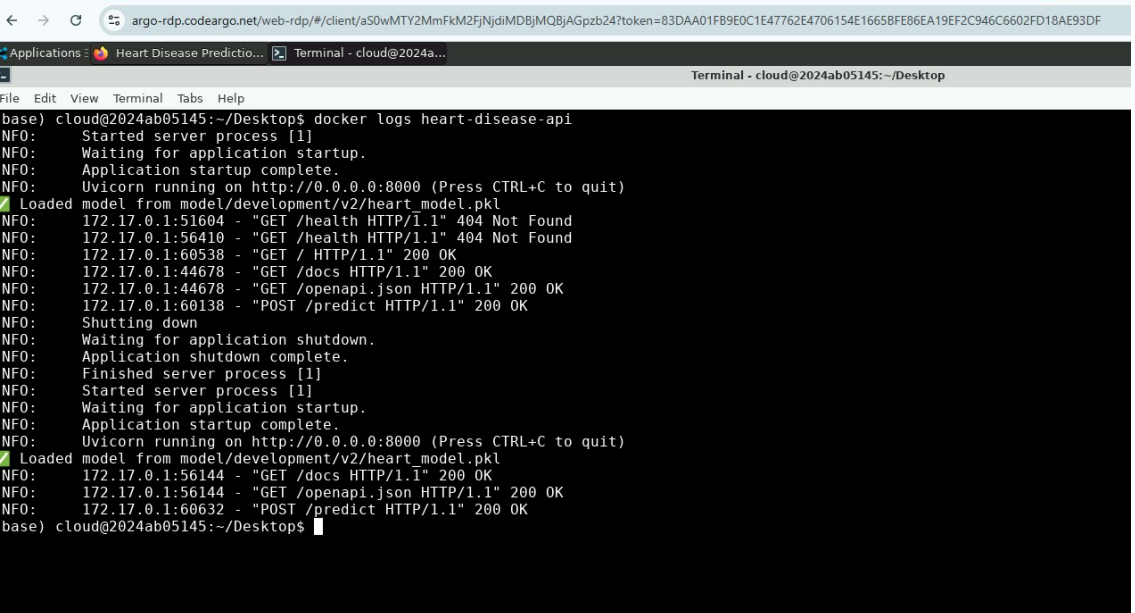
Deploy the Dockerized API to a public cloud or local Kubernetes (GKE, EKS, AKS, or Minikube/Docker Desktop).

Use a deployment manifest or Helm chart. Expose via Load Balancer or Ingress. Verify endpoints and provide deployment screenshots.



The screenshot shows the ArgoCD web interface for an application named 'Heart Disease Prediction'. The 'default' configuration is displayed, showing a 'POST /predict' endpoint. The request body is a JSON object containing patient data:

```
{
  "age": 63,
  "sex": 1,
  "cp": 1,
  "trestbps": 145,
  "chol": 233,
  "fbs": 1,
  "restecg": 2,
  "thalach": 150,
  "exang": 0,
  "oldpeak": 2.3,
  "slope": 3,
  "ca": 0,
  "thal": 6
}
```



The terminal window shows the deployment and testing of the 'heart-disease-api' using Docker and Uvicorn. The output is as follows:

```
base) ccloud@2024ab05145:~/Desktop$ docker logs heart-disease-api
INFO: Started server process [1]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
Loaded model from model/development/v2/heart_model.pkl
INFO: 172.17.0.1:51604 - "GET /health HTTP/1.1" 404 Not Found
INFO: 172.17.0.1:56410 - "GET /health HTTP/1.1" 404 Not Found
INFO: 172.17.0.1:60538 - "GET / HTTP/1.1" 200 OK
INFO: 172.17.0.1:44678 - "GET /docs HTTP/1.1" 200 OK
INFO: 172.17.0.1:44678 - "GET /openapi.json HTTP/1.1" 200 OK
INFO: 172.17.0.1:60138 - "POST /predict HTTP/1.1" 200 OK
INFO: Shutting down
INFO: Waiting for application shutdown.
INFO: Application shutdown complete.
INFO: Finished server process [1]
INFO: Started server process [1]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
Loaded model from model/development/v2/heart_model.pkl
INFO: 172.17.0.1:56144 - "GET /docs HTTP/1.1" 200 OK
INFO: 172.17.0.1:56144 - "GET /openapi.json HTTP/1.1" 200 OK
INFO: 172.17.0.1:60632 - "POST /predict HTTP/1.1" 200 OK
base) ccloud@2024ab05145:~/Desktop$
```

8. Monitoring & Logging [3 marks]

Integrate logging of API requests. Demonstrate simple monitoring (Prometheus + Grafana or API metrics/logs dashboard).

```
localhost:8000/metrics

# HELP python_gc_objects_collected_total Objects collected during gc
# TYPE python_gc_objects_collected_total counter
python_gc_objects_collected_total{generation="0"} 3727.0
python_gc_objects_collected_total{generation="1"} 345.0
python_gc_objects_collected_total{generation="2"} 6.0
# HELP python_gc_objects_uncollectable_total Uncollectable objects found during GC
# TYPE python_gc_objects_uncollectable_total counter
python_gc_objects_uncollectable_total{generation="0"} 0.0
python_gc_objects_uncollectable_total{generation="1"} 0.0
python_gc_objects_uncollectable_total{generation="2"} 0.0
# HELP python_gc_collections_total Number of times this generation was collected
# TYPE python_gc_collections_total counter
python_gc_collections_total{generation="0"} 381.0
python_gc_collections_total{generation="1"} 34.0
python_gc_collections_total{generation="2"} 3.0
# HELP python_info Python platform information
# TYPE python_info gauge
python_info{implementation="CPython",major="3",minor="10",patchlevel="0",version="3.10.0"} 1.0
# HELP http_requests_total Total HTTP requests
# TYPE http_requests_total counter
http_requests_total{endpoint="/metrics",method="GET",status="200"} 4.0
http_requests_total{endpoint="/favicon.ico",method="GET",status="404"} 2.0
http_requests_total{endpoint="/",method="GET",status="200"} 2.0
# HELP http_requests_created Total HTTP requests
# TYPE http_requests_created gauge
http_requests_created{endpoint="/metrics",method="GET",status="200"} 1.767634605661426e+09
http_requests_created{endpoint="/favicon.ico",method="GET",status="404"} 1.767634605775416e+09
http_requests_created{endpoint="/",method="GET",status="200"} 1.7676346492175791e+09
# HELP http_request_latency_seconds Request latency
# TYPE http_request_latency_seconds histogram
http_request_latency_seconds_bucket{endpoint="/metrics",le="0.005"} 0.0
http_request_latency_seconds_bucket{endpoint="/metrics",le="0.01"} 1.0
http_request_latency_seconds_bucket{endpoint="/metrics",le="0.025"} 1.0
http_request_latency_seconds_bucket{endpoint="/metrics",le="0.05"} 1.0
http_request_latency_seconds_bucket{endpoint="/metrics",le="0.075"} 1.0
http_request_latency_seconds_bucket{endpoint="/metrics",le="0.1"} 1.0
http_request_latency_seconds_bucket{endpoint="/metrics",le="0.25"} 1.0
http_request_latency_seconds_bucket{endpoint="/metrics",le="0.5"} 1.0
http_request_latency_seconds_bucket{endpoint="/metrics",le="0.75"} 1.0
http_request_latency_seconds_bucket{endpoint="/metrics",le="1.0"} 1.0
http_request_latency_seconds_bucket{endpoint="/metrics",le="2.5"} 1.0
http_request_latency_seconds_bucket{endpoint="/metrics",le="5.0"} 1.0
http_request_latency_seconds_bucket{endpoint="/metrics",le="7.5"} 1.0
http_request_latency_seconds_bucket{endpoint="/metrics",le="10.0"} 1.0
http_request_latency_seconds_bucket{endpoint="/metrics",le="+inf"} 1.0
http_request_latency_seconds_count{endpoint="/metrics"} 1.0
http_request_latency_seconds_sum{endpoint="/metrics"} 0.00801396369934082
http_request_latency_seconds_bucket{endpoint="/favicon.ico",le="0.005"} 2.0
http_request_latency_seconds_bucket{endpoint="/favicon.ico",le="0.01"} 2.0
http_request_latency_seconds_bucket{endpoint="/favicon.ico",le="0.025"} 2.0
http_request_latency_seconds_bucket{endpoint="/favicon.ico",le="0.05"} 2.0
http_request_latency_seconds_bucket{endpoint="/favicon.ico",le="0.075"} 2.0
http_request_latency_seconds_bucket{endpoint="/favicon.ico",le="0.1"} 2.0
http_request_latency_seconds_bucket{endpoint="/favicon.ico",le="0.25"} 2.0
http_request_latency_seconds_bucket{endpoint="/favicon.ico",le="0.5"} 2.0
http_request_latency_seconds_bucket{endpoint="/favicon.ico",le="0.75"} 2.0
http_request_latency_seconds_bucket{endpoint="/favicon.ico",le="1.0"} 2.0
http_request_latency_seconds_bucket{endpoint="/favicon.ico",le="2.5"} 2.0
http_request_latency_seconds_bucket{endpoint="/favicon.ico",le="5.0"} 2.0
http_request_latency_seconds_bucket{endpoint="/favicon.ico",le="7.5"} 2.0
http_request_latency_seconds_bucket{endpoint="/favicon.ico",le="10.0"} 2.0
http_request_latency_seconds_bucket{endpoint="/favicon.ico",le="+inf"} 2.0
```

```
argo-rdp.codeargo.net/web-rdp/#/client/aS0wMTY2MmFkM2FjNjdiMDBjMQBjAGpbz24?token=83DAA01FB9E0C1E47762E4706154E16658FE86A19EF2C946C6602FD18AE93DF

Applications : Heart Disease Predicto... Terminal - cloud@2024a...

Terminal - cloud@2024ab05145:~/Desktop

File Edit View Terminal Tabs Help

base) cloud@2024ab05145:~/Desktop$ docker logs heart-disease-api
NFO: Started server process [1]
NFO: Waiting for application startup.
NFO: Application startup complete.
NFO: Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
Loaded model from model/development/v2/heart_model.pkl
NFO: 172.17.0.1:51604 - "GET /health HTTP/1.1" 404 Not Found
NFO: 172.17.0.1:56410 - "GET /health HTTP/1.1" 404 Not Found
NFO: 172.17.0.1:60538 - "GET / HTTP/1.1" 200 OK
NFO: 172.17.0.1:44678 - "GET /docs HTTP/1.1" 200 OK
NFO: 172.17.0.1:44678 - "GET /openapi.json HTTP/1.1" 200 OK
NFO: 172.17.0.1:60138 - "POST /predict HTTP/1.1" 200 OK
NFO: Shutting down
NFO: Waiting for application shutdown.
NFO: Application shutdown complete.
NFO: Finished server process [1]
NFO: Started server process [1]
NFO: Waiting for application startup.
NFO: Application startup complete.
NFO: Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
Loaded model from model/development/v2/heart_model.pkl
NFO: 172.17.0.1:56144 - "GET /docs HTTP/1.1" 200 OK
NFO: 172.17.0.1:56144 - "GET /openapi.json HTTP/1.1" 200 OK
NFO: 172.17.0.1:60632 - "POST /predict HTTP/1.1" 200 OK
base) cloud@2024ab05145:~/Desktop$
```


9. Link to the code repository :

Git Hub Repository : <https://github.com/KSharma-SourceCode/AI-ML-Assignments/tree/main/M-Tech.%20Semester%203/MLOPS/Assignment-1/heart-disease-mlops>

Docker Hub Image Registry :

<https://hub.docker.com/repository/docker/ksharmadockerhub/heart-disease-api/general>

Deployment Manifests/Helm charts: <https://github.com/KSharma-SourceCode/AI-ML-Assignments/tree/main/deployment/helm/heart-disease>

Local Setup Instructions : <https://github.com/KSharma-SourceCode/AI-ML-Assignments/blob/main/M-Tech.%20Semester%203/MLOPS/Assignment-1/heart-disease-mlops/README.md>

CI/CD and Deployment Workflow : <https://github.com/KSharma-SourceCode/AI-ML-Assignments/actions>

Video Links:

YouTube: <https://youtu.be/ItkbTRBYdCc>

OR

Google Drive : <https://drive.google.com/file/d/1ZZHr5nZQmbYIZRzWCo-gNwi-Z2TJdB3e/view>